Journal of Intelligent Manufacturing
https://doi.org/10.1007/s10845-025-02740-z

Safe and efficient multi-agent planning for human–integrated smart manufacturing

Bassam Massouh[1] · **Fredrik Danielsson**[1] · **Sudha Ramasamy**[1]

Received: 6 July 2025 / Accepted: 10 November 2025

Abstract
Modern smart manufacturing requires flexible and safe coordination between human operators and autonomous agents. Current multi-agent system planning often relies on reactive safety mechanisms, leading to inefficiencies and workflow interruptions. Existing approaches typically overlook runtime safety policies during plan generation, producing functionally valid but operationally inefficient plans. As a result, avoidable slowdowns and emergency stops occur when humans enter shared workspaces, reducing both throughput and predictability. This research introduces a safety-aware planning approach that builds on an enhanced agent ontology by integrating agent negotiation with automated planning. A dynamic map of enhanced edges is constructed, where each edge represents a sequence of skills and its aggregated runtime execution cost. During runtime, agents negotiate to exclude unsafe edges and update costs based on runtime conditions, enabling the solver to generate plans that are both safe and efficient. Evaluation in a Plug & Produce case study with two system configurations demonstrated consistent advantages over a baseline reactive safety policy. The proposed planner, increased throughput by 50–80%, reduced execution costs by 20–55%, and lowered variability by up to a factor of 17. By integrating safety awareness into the planning stage, the approach improves efficiency, robustness, and compliance with human safety standards while supporting adaptable reconfigurable manufacturing. This work advances the vision of a human-centric future manufacturing by demonstrating that smart manufacturing systems can reason about operator presence to avoid hazards without compromising productivity.

Keywords Multi-agent system · Safety · Human · Automated planning

Introduction

Modern manufacturing strives to improve automation processes and realise Industry 4.0 (I4.0) goals by leveraging advanced technologies. Among these technologies, Multi-Agent Systems (MAS) are recognised as an effective solution to improve flexibility and performance of manufacturing (Vogel-Heuser et al., 2015). Industrial MAS controllers comprises intelligent software agents that collaborate to achieve the production goals (Leitão, 2009). A common approach is to assign high-level production goals to part agents. Resource agents, in turn, control physical equipment such as robots and machines (Farid & Ribeiro, 2015). To achieve their goals, part agents negotiate and coordinate with resource agents that offer capabilities in the form of executable skills (Kovalenko et al., 2023). These skills often have requirements that must be satisfied before execution, such as the part being positioned correctly or attached to a specific workstation (Nilsson et al., 2023). Fulfilling these requirements becomes a planning problem in which the system must identify the correct sequence that enables skill execution. Automated planning solves this planning problem by dynamically generating action sequences at runtime, removing the need for users to manually pre-program low-level device actions. This enhances manufacturing flexibility and allows system users to focus on higher value-adding tasks (Rogalla et al., 2018).

While significant progress has been made in applying automated planning to support flexible manufacturing (Marzia et al., 2023), most existing approaches still depend on reactive safety policies similar to those used in traditional automation systems (Hanna et al., 2022). Such approaches

✉ Bassam Massouh
bassam.massouh@hv.se

[1] Department of Industrial Automation, University West, Trollhättan, Sweden

Published online: 24 December 2025

do not adequately address runtime variability resulting from agents' autonomous behaviour. Hence, this study hypothesises that integrating runtime-evaluated safety policies into the automated planning process will yield more efficient transport and positioning sequences in human-integrated smart manufacturing environments than relying solely on reactive safety policies. Throughout this paper, autonomy refers to the agent's ability to negotiate and decide how to construct the planning domain and when to initiate the planning process. This capability enables agents to operate without external intervention in determining the prerequisites for planning. The planning process itself is commonly referred to as automated planning (Ghallab et al., 2016).

However, agent autonomy also raises important safety considerations. To ensure that agents' decisions remain safe in shopfloor environments, their behaviour must remain consistent with the implemented safety policies. In this context, safety policies are behavioural constraints enforced during runtime by controllers, such as safety Programmable Logical Controllers (PLCs), to ensure safe interactions between resources and human workers. This research aims to bridge the gap between the MAS and shopfloor safety by enabling agents to understand safety policies and plan according to the safety needs. At the same time, it addresses the agent collaboration complexities in MAS-controlled manufacturing environments. MAS environments involve multiple autonomous agents executing different skills concurrently, which increases the risk of conflicting or unsafe actions (Rizk et al., 2018). To manage these conflicts, research in MAS scheduling has developed mechanisms such as path planning and temporal coordination, widely applied in domains like autonomous vehicles in terms of path finding and task allocation (Alkazzi & Okumura, 2024; Choudhury et al., 2022; Zhou et al., 2022), and robotic swarms (Shibuya et al., 2023). These methods prevent agents from colliding and occupying the same space. However, their application into manufacturing safety remains limited, focusing mainly on autonomous vehicle collisions. Therefore, a coordination mechanism is needed to address hazards arising from overlapping workspaces, especially where humans are involved in manufacturing settings and to ensure compliance with the relevant safety standards such as ISO 10218 (SIS (Swedish Standards Institute), 2025) and ISO/TS 15066 (SIS (Swedish Standards Institute), 2016).

Despite notable advances of MAS in manufacturing, a critical gap remains, current systems lack the ability to proactively incorporate runtime safety conditions during plan generation. As a result, the plan's action sequence is often functionally valid but operationally inefficient, triggering avoidable slowdowns or emergency stops due to unanticipated human–system interactions. This research addresses this gap by embedding runtime safety reasoning directly into the planning process, shifting safety from a reactive enforcement mechanism to a context-aware decision-making feature.

Extending the safety-aware Configurable Multi-Agent System (C-MAS) ontology introduced in (Massouh et al., 2024), this work contributes a novel planning approach where agents negotiate runtime conditions, allowing the planner to avoid unsafe sequences and account for execution costs during runtime. It generates transport and positioning plans by excluding unsafe operations, selecting the least-cost sequences, and coordinating concurrent skills to avoid hazardous overlaps. This enables safe and efficient automated planning, particularly in human-integrated smart manufacturing environments. In addition to the planning approach, this work enhances the underlying safety-aware MAS ontology, enabling it to support automated planning. The enhanced ontology provides the semantic foundation for hierarchical skill modelling, recursive planning, and agent collaboration that adapts safely to runtime conditions.

The rest of this paper is structured as follows: Sect. "Background" describes the background, including MAS and existing approaches to planning and safety in manufacturing. Sect. "MAS architecture and ontology" presents the enhanced safety-aware C-MAS ontology that enables the proposed planning approach. Sect. "Map construction and planning problem formulation" details the planning problem formulation. Sect. "Agent negotiation for remote skills allocation" describes the agent collaboration mechanism. Sect. "Solver call and recursive execution" provides a compilation of the planning approach that leverages the planning problem formulation and agent negotiation to generate safe and efficient plans. Sect. "Evaluation of the planning approach" evaluates the approach through a Plug & Produce case study, discusses the approach's scaling complexity, simulation results, and statistical analysis. Finally, Sect. "Conclusion" concludes the paper and outlines directions for future work.

This paper scope is reconfigurable manufacturing systems controlled by a multi-agent system, with a specific emphasis on integrating human operators into smart cells while ensuring their safety during automated planning. The planner is explicitly designed for cell-level control, which aligns with industrial safety practices where typically safety PLC policies are engineered and certified per cell. We do not attempt to generate factory-wide plans in a single pass; instead, larger systems composed of multiple cells reuse the same multi-agent pattern, preventing the planning problem from growing without bound. Within each cell, agent negotiations are handled locally, where concurrent negotiation requests may occur but are resolved on a first-come–first-served basis. Retry and backoff policies ensure progress, fairness and livelock avoidance. No global coordination or

priority scheduling across multiple cells is included, as optimisation across competing products falls under higher-level job-shop scheduling. Validation is conducted in a simulated Plug & Produce environment under two configurations, one with a single-workstation active and another extended setup enabling parallel execution. Three human presence patterns are also considered: low, medium, and high frequency of operator entry into the shared workspace. The cell design inherently minimises large-scale scheduling complexities while allowing to evaluate the effectiveness of the proposed safety-aware planning approach.

Background

The concept of a "smart factory" is a cornerstone in the vision of I4.0. The term is introduced within the strategic initiative Industrie 4.0, which envisioned a "smart connected world" with a smart factory at its centre (Kagermann et al., 2013). At the heart of this vision is cyber-physical integration realised through Cyber-Physical Systems (CPS). The concept of CPS was formally introduced by A. Lee in his influential paper *"Cyber-Physical Systems: Design Challenges"* (Lee, 2008). CPS design imposes requirements such as adaptability, interoperability, and safety/security, all of which are essential for modern manufacturing systems (Gunes et al., 2014; Ribeiro & Hochwallner, 2018). Meeting these requirements using traditional automation is often infeasible. As an alternative, MAS has gained traction as an effective solution for enabling the decentralisation, responsiveness, and flexibility required in CPS-based manufacturing environments (Vogel-Heuser et al., 2015). In MAS-controlled manufacturing, agents are assigned to control shopfloor resources. Each agent acts autonomously and collaboratively to manage decision-making, enabling real-time adaptability (Arai et al., 2003).

To realise MAS in practice, several reference architectures have been developed, including PROSA, ADACOR, HCBA, and RAMI 4.0, each offering a structured approach to modelling system components and interactions (Kaiser et al., 2023). One of the notable implementations of MAS in manufacturing is the IDEAS project, which demonstrated successful shopfloor reconfiguration using parts and resources agents (Onori et al., 2012). A more recent advancement in multi-agent control, which is inspired by the IDEAS project, is the C-MAS, which implies that all agents are configured rather than programmed. C-MAS shortens engineering time and reduces the level of needed competencies (Nilsson et al., 2023).

This article builds on the ongoing work of C-MAS. Its conceptual framing is grounded on the established model-based agent principles for smart manufacturing, where each agent, whether a part or a resource agent, maintains an internal model of its counterpart and the manufacturing environment. The model-based agents enable reasoning beyond reflexive behaviour. Part agents are goal-based, equipped with algorithms and logic that allow them to plan sequences of actions to achieve manufacturing objectives (Kovalenko et al., 2023). Resource agents in turn use internal models to evaluate their own capabilities and resource state (Bi et al., 2024). These representations are derived from a shared ontology. At the system level, C-MAS employs a shared ontology to serve as the global domain model, defining entities, relationships, and constraints across agents. Ontology-based MAS concept has proven effective in manufacturing contexts for enabling scalable interoperability and knowledge-driven planning (Radakovič et al., 2012).

Service-oriented tools have been developed to facilitate the manual orchestration and planning of resources agents jobs (Kavvathas et al., 2024). However, in C-MAS, automated planning is employed to fulfil the skills requirements before they can be executed. The planner searches for the most efficient and safe path from the current system state to the required condition. This is particularly relevant for transport and positioning, which are critical for enabling skill execution but tedious to program manually. Automated planning is a branch of artificial intelligence focused on developing strategies that generate action sequences that enable an agent or system to reach specific goals from a given initial state. Unlike reactive systems, automated planning involves deliberation where an "actor" decides what actions to take and in what order to reach a desired outcome (Weiming Shen et al., 2006). In MAS-controlled manufacturing, automated planning allows agents to autonomously generate action sequences to meet skill requirements, reducing manual programming and enabling a more flexible system behaviour. Advances in automated planning have enhanced robotic agility, supporting dynamic task reallocation and failure recovery (Kootbally et al., 2018), automatic domain model generation for flexible assembly tasks (Heuss et al., 2024), and ontologies for storing reusable primitive actions to reduce re-planning needs (Qian et al., 2023). However, modern skill models often embed high-level programming languages, such as Structured Text (ST), granting skills internal logic, conditional flows, and nested skill calls with dynamic requirements (Lyu & Brennan, 2024). This work builds on those advances by recognising that embedded logic within skills introduces additional planning complexity, skills can no longer be treated as atomic actions. Unlike purely orchestrated MAS, where high-level coordination is managed by the MAS and low-level logic is executed by shopfloor components, embedding logic directly within skills increases flexibility at the MAS level but also leads to a significantly more complex planning problem.

Moreover, the integration of humans into smart manufacturing introduces the challenge of balancing operational efficiency with worker safety, an area that has only recently begun to receive significant attention (Kheirabadi et al., 2023). Much of the existing work addresses this within Human–Robot Collaboration (HRC) by optimising task distribution. For example, (Qu et al., 2024) provides a method suitable for optimizing and safely executing HRC tasks primarily through task allocation and monitoring within a digital twin. Also, (Pupa & Secchi, 2021) presents a centralised HRC planner that enforces safety at execution via Speed and Separation Monitoring (SSM) and reschedules on failure. Similarly, (Papavasileiou et al., 2024) propose a centralised AI-based framework that combines task planning with digital simulation and dynamic reconfiguration to adapt to unexpected events. These are not sufficient for the distributed nature of a system of autonomous agents. Recent reactive motion planning methods include collaborative-robot path updates that continuously adjust trajectories in real time to maintain safe separation from humans and obstacles (Scholz et al., 2025). Other approaches rely on supervisory filters that dynamically reshape robot commands during execution to enforce constraint compliance (Merckaert et al., 2024). These approaches function solely at the execution phase. By contrast, the proposed approach proactively identifies and eliminates unsafe transitions and incorporates cost adjustments during the planning phase, before solving the planning problem. Human-aware task planners that replan on operator triggers or switch collaborative modes at decision points (Gottardi et al., 2023; Ma et al., 2025) focus on single robot–human pairs, whereas this work coordinates an entire cell-level MAS of part and resource agents. While such approaches represent important advances, they do not account for how the implications of runtime safety policies may influence overall planning efficiency. Similarly, other methods address safety in a reactive manner. For example, applying adaptive path adjustment during execution to maintain a safety distance from obstacles, rather than a proactively accounting for the performance impact of the unsafe actions in the planning phase, particularly with respect to safety policies related to human presence or workspace overlaps (Hu et al., 2023). Other work utilising deep reinforcement learning for disassembly plans to optimises energy use and recovery profit but left applying multi-agent system to balance efficiency and HRC to future work (Wang et al., 2025). Zhao et al., (2025) advanced this by integrating safety-efficiency trade-offs based on human–robot separation, yet their method lacks consideration of skill internal logic and does not extend to system-level MAS coordination.

Critically, safety and efficiency must be addressed not just at the level of individual robotic tasks but across the entire manufacturing system. Research into system-level planning for manufacturing involving complex skills is still sparse. Recent studies have explored hierarchical production planning under retail uncertainty (Deng et al., 2025) and MAS-based self-management using IEC 61499 function blocks (Lyu & Brennan, 2024). Still, these approaches fail to ensure proactive worker safety within smart manufacturing systems. Current MAS safety strategies remain focused on cybersecurity, fault tolerance (Tsutsumi et al., 2023), or reactive measures rather than embedding proactive, efficiency-aware risk avoidance into planning processes (Gao et al., 2022; Ju et al., 2022; Zhang et al., 2021).

Despite advances in flexible task planning, today's MAS solutions still handle worker safety as a fixed rule enforced after plans generation rather than as a dynamic factor in planning process. This limits their ability to adapt to real-time safety conditions, especially in systems with concurrent agents' actions and embedded skill logic. To address this gap, this paper introduces a planning approach that integrates runtime agent negotiation into planning, enabling agents to evaluate both the safety and cost of operations before execution. This approach supports safer and more efficient planning in smart, human-integrated manufacturing environments.

MAS architecture and ontology

This section presents the ontology developed to support automated planning of transportation and positioning requirements in C-MAS with focus on workers' safety. In this paper, the ontology refers to the C-MAS metamodel defining the structure and domain semantics across various applications. An application model is an instance of this ontology that is configured at design time. At runtime, the application model's planning semantics, together with agent behaviour e.g., agent negotiation and local policies, produce the needed plan, rather than any ontology-based inference.

The ontology enhances the original C-MAS architecture introduced in earlier work for risk-aware control of reconfigurable manufacturing systems (Massouh et al., 2024). The foundational structure includes several core classes: *Agent, Variable*, Goal, Interface, Skill, and Hazard. All abstract classes are written in italics. Resource and Part agents are subclasses derived from an abstract *Agent* class. Each class within this ontology derives from an abstract base class called *Entity*, providing basic metadata attributes like instance name and unique identifiers. There exist several specialised variable classes that store configuration and runtime data and extend from a general abstract *Variable* class. Table 1 summarises the core classes in earlier work.

Table 1 Description of the core classes of the safety-aware MAS controller as presented in (Massouh et al., 2024)

Class	Description
Agent	Represents an autonomous entity in the system that can be a part or a resource with name and unique instance ID properties
Variable	Encapsulates data relevant to decision-making within the MAS
Goal	Defines the objectives that a part agent must fulfil during the execution of manufacturing processes
Interface	Captures the physical and logical compatibility between agents. It serves as a boundary for agent interaction, characterised by a set of skills and variables. Interfaces are used to assess whether two agents can collaborate, e.g., whether a robot can physically handle a part or tool, and if the logical interface matches the control variables
Skill	Represents a specific executable functionality provided by an interface on a resource agent. A skill contains logic that can include both local actions and remote calls and is associated with zero to many hazards and requirements
Hazard	The Hazard class encapsulates the risk level and the hazard target type, which specifies the type of resource a hazard of a skill affects, e.g., human operator or machine
Risk	Aggregates hazard-related data to inform safety evaluations

To enable automated planning, this work introduces three new classes, $WorkFlow$, $Requirement$ and $InterfaceSpecialisation$ and one enumerated list WorkFlowType. The enumerated list WorkFlowType includes standard primitives used to define both the requirements and postActions of skills. The list values include ATTACHED, DETACHED, and TRANSFERRED, which are relevant for transport and positioning operations. More workflow types exist but they are out of the scope of this paper. The $WorkFlow$ class, which inherits from $Entity$, encapsulates a single WorkFlowType. The $Requirement$ class inherits from $WorkFlow$ class and encapsulates a Target attribute, indicating the agent to which the requirement applies.

Additional enhancements to the core classes include modification of the Skill class, which includes attributes PostAction, Cost, and HazardTargetType. PostAction, of the WorkFlowType, are properties associated with the skill, specify the resulting state after a skill has been executed. The HazardTargetType property specifies the type of resource a hazard may affect, such as a human operator or a machine. This distinction ensures that hazards are only evaluated for relevant resource types, as a hazard might pose a risk to one type but not impact others. The property Cost represents a generic cost value associated with performing the skill. This value can be a predefined static value or dynamically determined through negotiation at runtime. Additionally, the class Skill has association relationships with zero or more instances of Hazard and $Requirement$.

The class $Interface$ models both logical and physical compatibility between agents. Interface compatibility is determined based on shared names and matched variable values. This variable matching goes beyond simple equality. It supports complex patterns, value ranges, and conditional rules, enabling more expressive and flexible compatibility checks between agents. Skills have a composition relationship with an interface, specifying that each skill is bound to the specific physical and logical capabilities of its associated interface. To call for specific interface instant in a skill logic, abstract interfaces are used. The term “abstract interface” is a generic handle for invoking a skill without specifying a resource. At runtime it binds, through negotiation, to a compatible “remote interface” on an available resource. An additional new class InterfaceSpecialisation is added to the ontology. It extends the $Interface$ class and is used to include information that goes beyond general compatibility between agents. The data within these interfaces, called SpecialisationVariables, are not part of the standard matching process during agent negotiation. This means agents don’t use them to decide whether they can work together but rather gives extra information about the interface that can be used. For example, a specialisation variable might specify the physical location of a workstation or robot arm, which is crucial when planning the transportation and positioning of agents. For transport and positioning planning, two relevant InterfaceSpecialisation templates are defined; location and transport specialised interfaces. These specialised interfaces hold skills that are used directly in planning. These skills have postActions ATTACHED and/or DETACH on location interfaces and TRANSFERRED on transport interfaces. The planner therefore reasons in terms of the next required postAction on named specialised interface, while the skill execution handles the detailed skill logic. This separation is deliberate to address the flexibility trade-off. Abstraction at the planning level reduces the state space, and flexibility is preserved because the fine-grained behaviour remains inside the skills. Figure 1 shows the class diagram of the enhanced C-MAS ontology for automated planning and safety negotiation in MAS-controlled manufacturing.

The ontology distinguishes between goals and requirements based on their complexity. Requirements are simple atomic conditions, such as verifying that a part is attached to a particular resource, while goals entail complex sequences involving multiple skills. For example, a part agent may define a sequence combining requirements and goals, where fulfilling a requirement, e.g., attaching with another agent, triggers the planner to generate a corresponding operational sequence. In contrast, goals require predefined logical structures or skills to identify suitable execution strategies.

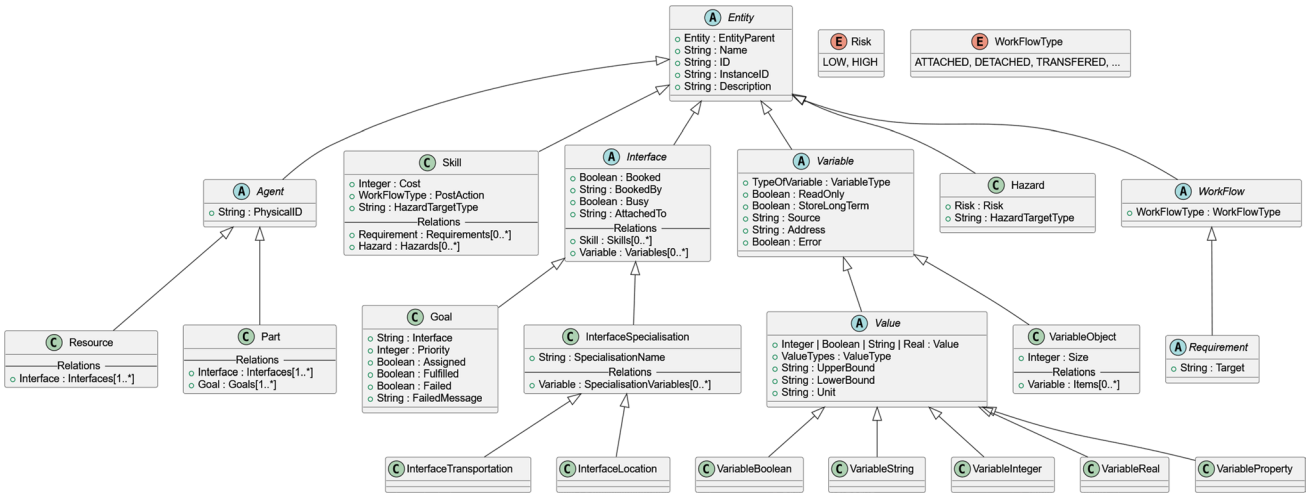

Fig. 1 Class diagram of enhanced C-MAS ontology for planning and safety negotiation in MAS-controlled manufacturing. Class types: A—abstract, C—concrete, E—enumerated

Map construction and planning problem formulation

In automated planning, problem domains are often modelled as graphs of nodes and edges. Nodes represent system states and encode propositions about the system at a given moment, while edges correspond to transitions from one state to another. Each edge typically has an associated cost, reflecting metrics such as time, energy, or operational risk. The planner's objective is to identify a sequence of edges from an initial state to a goal state that minimises the total cost.

This work builds on this classical representation of Graph plan introduced by (Blum & Furst, 1997) and adapts it for the C-MAS architecture. In Graph plan, the planning domain actions are modelled as transitions and each transition moving from one state to another. To address this, we introduce a map of enhanced edges. An enhanced edge is a macro-transition defined over three interface specialisations: a source (location specialisation l_k), a transport (transport specialisation τ_l), and a target (location specialisation l_m). It defines a sequence of skills that move an agent ag from a source interface to a target interface via a transport interface. This modelling approach significantly reduces the planning state space and aligns with industrial practices, where a sequence of low-level skills are grouped into modular blocks to reduce complexity. At the same time flexibility and granularity at skills level is still preserved. Conceptually, this relates to the long-standing use of macro-transition/macro actions used to increase the speed of the planner (Botea et al., 2005; Coles & Smith, 2007). The enhanced edges differ as they are constructed, by interfaces compatibility, from the application model instantiated at the design time from the C-MAS ontology. Sources hold skills with postAction DETACHED, similarly transports hold TRANSFERRED, and targets hold ATTACHED. Also, before solving, interfaces owning the skills perform safety assessment negotiation to assign a runtime cost and prune unsafe edges, whereas prior formulations of a macro transition don't include safety assessment and costing at the graph construction time.

Agents with compatible interfaces exchange information about safety, availability, and execution conditions, returning an enhanced edge with a unique identifier $uid\,(e)$ and a computed total cost $Cost\,(e)$. The total cost of an edge is defined as in (1),

$$Cost\,(e) = c_{detach}\,(e) + c_{transport}\,(e) + c_{attach}\,(e) \tag{1}$$

where, $c_{detach}\,(e)$ is the execution cost of detaching from the source interface, $c_{transport}\,(e)$ is the execution cost of transport via the transport interface, and $c_{attach}\,(e)$ is the execution cost of attaching to the target interface.

The map is then defined as the 2-tuple in (2),

$$M = \langle S, E \rangle \tag{2}$$

where:

- S is the set of location specialised interfaces in the system,
- E is the set of enhanced edges, each $e \in E$ defined as in (3), which connects a source and target interface through a transport interface.

$$e = \langle \ell_k, \tau_l, \ell_m, uid\,(e)\,, Cost\,(e) \rangle \tag{3}$$

We distinguish between the map M, which encodes all feasible operations as enhanced edges and the planning problem P, which instantiates the map with the current system state and a goal or requirement to be fulfilled.

Internally, the planner models the system state as a vector of the location interfaces, where each interface can either be free or occupied by a specific agent. The transport interface τ_l is embedded within the edge and therefore does not appear explicitly in the state space. This abstraction allows for retaining a compact solver model without sacrificing the accuracy of cost minimisation. A state $s \in S$ is encoded as is (4),

$$s = (N_1, N_2, \ldots, N_k), N_{inf} \in \{\varnothing\} \cup Ag \tag{4}$$

where:

- $N_{inf} = \varnothing$ indicates that the interface inf is free,
- $N_{inf} = ag \in Ag$ indicates that it is occupied by an agent ag

Then the planning problem is defined as in (5)

$$P = \langle M, s_i, S_g \rangle \tag{5}$$

where:

- $s_i \in S$ is the initial state; the runtime attachment status of all locations specialised interfaces at the time of planning.
- $S_g \subset S$ is the set of goal states; the target attachment status of all locations specialised interface that satisfies the positioning requirement.

Once the planning problem is specified, it is compiled into a model suitable for the solver. In the new model, each enhanced edge becomes a transition, labelled with its $uid\,(e)$ and associated with its negotiated cost. Since the cost already accounts for the transport component, the solver can find the minimum cost path without considering transport interfaces as separate nodes. The objective of the solver is to find a valid plan $\pi = [e_1, e_2, \ldots, e_n]$, such that the system transitions from the initial state s_i to a goal state $s_g \in S_g$ along the sequence of transitions: $s_i \overset{e_1}{\rightarrow} s_1 \overset{e_2}{\rightarrow} \ldots \overset{e_n}{\rightarrow} s_g$ while minimising the total cost:$Minimize \sum_{e \in \pi} Cost\,(e)$

The result returned by the solver is a sequence of edge identifiers. These are referenced against the map M to reconstruct the full skill sequences for execution, including the intermediate transport interface. The resulting plan is then dispatched to the agents for execution.

Agent negotiation for remote skills allocation

Agent negotiation in C-MAS is aligned with the established FIPA Contract Net Protocol (CNP), a well-established coordination mechanism in MAS. The negotiation begins with an agent issuing a request Call For Proposals (CFP) to potential contractors using the standard FIPA Agent Communication Language (ACL). Agents with compatible interfaces respond with proposals, and the requester awards the skill by sending an accept-proposal, while dismissing others with reject-proposal and forming binding commitments upon acceptance. Contractors then complete the skill and notify the requester via an inform or a failure if they cannot execute it.

The collaboration process between agents to safely and efficiently allocate and execute remote skills is modelled as a sequence diagram in Fig. 2 and is conducted through the following sequential steps for all local interfaces. A local interface refers to the interface that request and initiates the execution of a skill. A local skill can include calls to other skills on remote interfaces. From the perspective of the local interface, the remote interface is the other interface involved in the negotiation and execution. The identity of the remote interface is not known in advance i.e., abstract interface, and is resolved at runtime when needed through negotiations:

1. Query all remote agents for interfaces compatible with the local interface and obtain current status. Build the negotiation message payload (demanded skill+interface variables/constraints), attach a unique request_id, and set a *reply-by* time. Remove all interfaces that are busy or booked. This step filters out interfaces not currently available for negotiation.
2. Dispatch an ACL CFP to all compatible remote interfaces, enclosing the payload and *reply-by* timestamp that specifies the latest acceptable time for responses. This triggers the onNegotiate() call on each remote interface.
3. Each remote interface evaluates availability, safety, and cost and replies *propose* with skill cost≥0 or *refuse* if busy or unsafe. Thus, busy or unsafe interfaces are filtered by *refuse*.
4. Collect *propose/refuse* messages until the *reply-by* time. Late replies may be ignored.
5. From the proposals, select the lowest-cost one. To break exact ties deterministically, use first-come-first-served at the resource. Send *accept-proposal* to the winner; send *reject-proposal* to the others. After acceptance, the contractor is committed to perform the task.
6. If the selected skill has a high-risk hazard, send an ACL *query-if overlap?* to all remote interfaces in the cell/domain. The interfaces reply with *overlap=true* if a hazardous spatial conflict exists, or *overlap=false* if it does not.
7. For every interface replies with *overlap=true,* send ACL *request* to book/lock it for execution window. The overlapped interfaces *agree* and then *inform* once locked. This prevents other agents from using it and avoid unsafe special overlaps.
8. The winning interface *confirms* booking and executes the remote skill; on completion it sends *inform* done/result

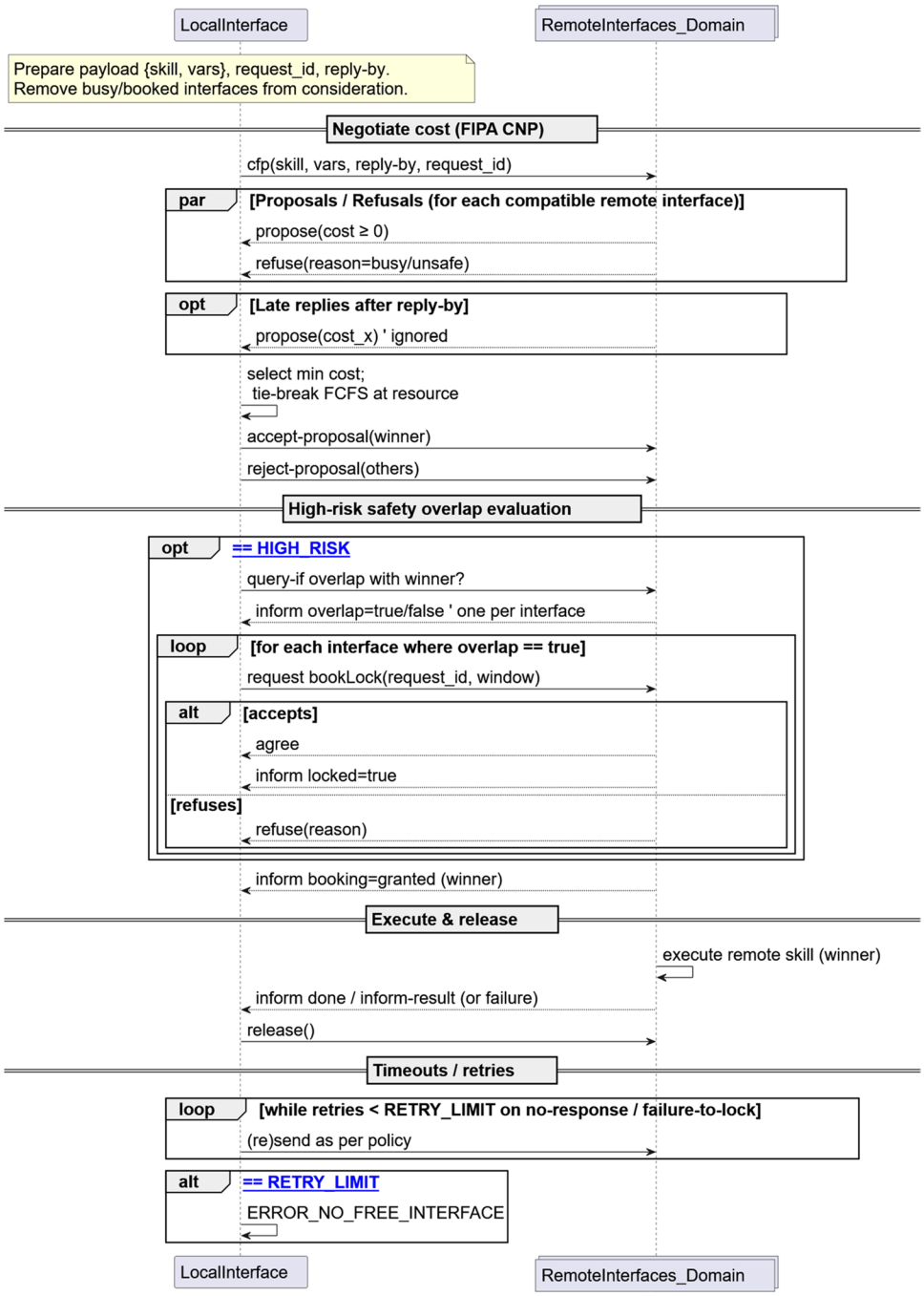

Fig. 2 Sequence diagram for agent collaboration

or *failure*. Release all bookings/locks. If confirmations or results time out, retry according to policy; if the retry limit is reached without success, throw an error and stop.

If negotiation is successful, the interface with the lowest skill execution cost is selected. This interface is then reused for any subsequent remote actions referencing the same abstract interface, ensuring consistency throughout the execution flow. If it fails, the number of retries is limited by user defined settings. Then it gives an error to the user. If negotiation fails because no free or safe interface is found, the requesting agent re-issues the request up to a user-defined limit, which may be set to indefinite. Failure is reported if the retry limit is reached without a successful negotiation.

Solver call and recursive execution

The proposed planning approach leverages the planning problem formulation presented in Sect. "Map construction and planning problem formulation" and the agent negotiation in Sect. "Agent negotiation for remote skills

allocation". The approach has a hierarchical structure from goals to execution. A part agent's goal trigger execution of skills and when skills have requirements they trigger the planning which comprises of sequence of skills following a hierarchical structure.

This hierarchical implementation is illustrated using high-level abstraction pseudocodes. These pseudo-codes capture the core logic, whereas the actual system can handle everything from basic action lists to advanced programming constructs, including complex structures, iterations, conditionals, and decision-making logic. The implementation is organised around two main functions: $fulfilRequirement$ in Pseudocode 1 and $executeSkill$ in Pseudocode 2. The function $fulfilRequirement$ initiate by capturing the initial state for the solver using the method $getInterfacesAttachmentStatus$, which retrieves the attachment status of all specialisation interfaces of type location within the domain. It then identifies the desired goal state based on the requirements specified by the demanded skill.

To construct the map of enhanced edges, the function $generateEnhancedEdge$ is invoked. This function receives as input the domain, which includes all agents within the C-MAS application model. It begins by identifying all compatible interface specialisations of type location and enumerate them into two lists: the $fromList$, containing interfaces suitable as source locations, and the $toList$, containing interfaces suitable as target locations. Interfaces are classified into the $fromList$ if they include at least one skill with the postAction DETACHED, indicating readiness for pickup from. Conversely, interfaces are included in the $toList$ if they have a skill with the postAction ATTACHED, indicating readiness to place onto. Additionally, the function identifies all compatible interface specialisations of type transport. These are compiled into a $transportList$ and are characterised by containing at least one skill with the postAction TRANSFERRED.

$Negotiation()$ function follows the negotiation for cost presented in Sect. "Agent negotiation for remote skills allocation". The agent that initiated planning, the requester, issues a CFP per skill to the owning agent. Upon receiving a CFP, the agent executes a user-defined onNegotiation() method, specified at design time, which applies safety logic e.g., processing live signals, consulting a rule base, or invoking a learned policy. The agent then replies either with a runtime, safety-adjusted cost, or REFUSE. If any agent replies REFUSE, the entire edge is discarded for that planning process. Otherwise, the returned costs are aggregated into a safe, weighted edge. As a result, the solver therefore operates only on safe, negotiated edges that already include their runtime costs.

With the safe map constructed, a planning problem is formulated, and a solver then identifies the least-cost sequence of enhanced edges to fulfil the requirement. The resulting list of edge identifiers (planUIDs) is used to retrieve the corresponding enhanced edges. The skills in the edges are then executed in order using the $executeSkill$ function.

Pseudocode 1 describes the logic for requirements fulfilment, including the generation of the planning problem, the solver plan, and the execution of the plan

```
function fulfilRequirement(skill, domain)
  initailStates := getInterfacesAttachmentStatus(domain.getAllInterfaces())
  goalState := skill.requirement
  enhancedEdges := generateEnhancedEdge(domain)
  for each edge in enhancedEdges do
    if Negotiation(edge.skills) then
      map := map + edge
    end if
  end for each edge
  problem := generateProblem(map, initialStates, goalState)
  planUIDs := solveForLeastCostPlan(problem)
  for each uid in planUID do
    plan := plan + lookup(map, uid)
  end for each uid
  for each skill in plan do
      executeSkill(skill, domain)
  end for each skill
  return TRUE
end function
```

Skills are executed through a dedicated skill execution function, $executeSkill$ that supports hierarchical skill execution while ensuring runtime safety. If a skill is high risk in the application model, the function first invokes $getAllOverlappingInterfaces$ to identify any overlapping interfaces that may be affected by the skill's hazard. The onOverlap() method is invoked on all other remote interfaces. Defined during the design phase, this method checks for spatial conflicts by evaluating whether the hazard associated with the selected skill affects any other resource that is specified by the HazardTargetType property. It then determines whether the operational zone of such a remote skill overlaps with that of the selected skill. If an overlap is detected, these interfaces are then booked before the skill is executed, preventing unsafe spatial conflicts during runtime and released when the skill is done. Book has a limit of tries and when it reaches the limit identified by the user, without booking all overlapping interfaces, it sends an error that interfaces cannot be booked.

If the selected skill has its own requirements, the function recursively calls $fulfilRequirement$ to ensure those are satisfied before proceeding with execution. Once all actions and skills are completed, any previously booked resources are released. Both $executeSkill$ and $fulfilRequirement$ return true upon successful completion or false if they fail.

$executeSkill$, can fail for three reasons: (i) inability to book overlapping interfaces for safety, (ii) failed negotiation for abstract interfaces, or (iii) unsatisfied

requirements. In these cases, the function returns *false* and calls $executionErrorHandling()$, which may involve operator intervention or replanning. $fulfilRequirement$ may also fail if negotiation for compatible interfaces is unsuccessful after the defined number of retries. In such cases, the candidate interface is removed from the list, and the function returns *false*.

During negotiation, the agent waits until either (i) all replies have been received or (ii) the reply-by deadline has expired, with late replies are ignored. While this may delay progress and affect responsiveness in the presence of communication issues, reliability remains unaffected since no action is taken until all required replies are collected, ensuring that unsafe behaviour cannot occur. The planning approach is implemented on cell level with dedicated network infrastructure, where message exchange times are negligible compared with solver and execution times, making latency effects minimal. Under timeout conditions, the system degrades gracefully either by reporting the failure or reissue the messages.

Pseudocode 2 The logic for executeSkill that supports local and remote action while ensuring runtime safety

```
function executeSkill(skill, domain)
  if skill.hazard = HIGH then
    overlappingInterfaces := getAllOverlappingInterfaces(skill, domain)
    wait until book(overlappingInterfaces)
  end if
  for each action in skill do
    if action is local then
      executeAction(action)
    else if action is remote then
      candidateInterfaces = findCompatibleInterfaces(action)
      for each interface in candidateInterfaces do
          if not negotiateIntrface(interface) then
            remove interface from candidateInterfaces
          end if
      end for each interface
      if candidateInterfaces is empty then
          return FALSE
      end if
      skill := selectLeastCostSkill(candidateInterfaces)
      for each requirement in skill.requirements do
          if not fulfilRequirement(skill, domain) then
            return FALSE
          end if
      end for each requirement
      if not executeSkill(skill) then executionErrorHandling()
    end if
  end for action
  Unbook(overlappingInterfaces)
  return TRUE
end function
```

Evaluation of the planning approach

The proposed planning approach is evaluated using a Plug & Produce case study. The evaluation demonstrates the approach's practical utility in real-world manufacturing system. The case study has been developed in collaboration with industrial partners as part of ongoing research projects. It supports runtime reconfiguration, allowing the addition or removal of resources, making it a representative environment for exploring the feasibility of adaptive and safety-aware planning strategies. Figure 3a Shows the Plug & Produce cell the case study is based on and Fig. 3b shows the labelled cell layout.

To ensure human safety, the cell is equipped with safety scanners that monitor operator positions in real time. Robot behaviour adheres to the SSM principle as defined in ISO/TS 15066 (SIS (Swedish Standards Institute), 2016). This principle dynamically adjusts robot speed based on the distance to nearby human workers. Three speed zones were defined: when the separation distance exceeds 1.5 m, the robot operates at its maximum speed of 500 mm/s; between 1.2 and 1.5 m, the speed is reduced to 150 mm/s; and when the distance is less than 1.2 m, the robot halts. These thresholds were calculated based on the dynamic SSM equation presented by (Byner et al., 2019)

$$v_{max} \leq \sqrt{v_h^2 + (a_s T_r)^2 - 2a_s (C - S)} - a_s T_r - v_h$$

Where the decided values are $v_h = 1.6m.s^{-1}$ is human movement speed, $a_s = 15m.s^{-2}$ is the ABB 6700 robot deceleration, $T_r = 0.2s$ is the system reaction time, $C = 1m$ is additional safety margin, and s is the separation distance between the operator and the robot.

The Plug & Produce layout comprises a source, two workstations WS_1 and WS_2 each have one location for assembly and one location for inspection, and two intermediate buffer stations (B_1 and B_2). The system can operate in two configurations: Configuration A, where only WS_1, B_1 are active, and Configuration B**,** where both workstations and buffers are active, enabling parallel execution.

Multi-agent system configuration

The agent configuration used in the validation scenario follows the C-MAS ontology. The scenario centred on producing a part agent that is initially attached to the source station and has the goal of *Assembly*. This goal is resolved at runtime by invoking a composite skill called *assembly*, The *assembly* skill is modelled hierarchically and internally calls the skill *inspection*. This demonstrates how a goal that appears simple at design time unfolds into a complex chain of planning and negotiation steps involving hazard-aware decisions and runtime coordination.

The stationary resources *source*, *buffers*, *workstation_assembly*, and *workstation_inspection*, all configured with location interfaces with variables indicating their locations and skills for ATTACHED and DETACHED postActions. The resources *workstation_assembly*, and *workstation_inspection*, have the skills *assembly* and

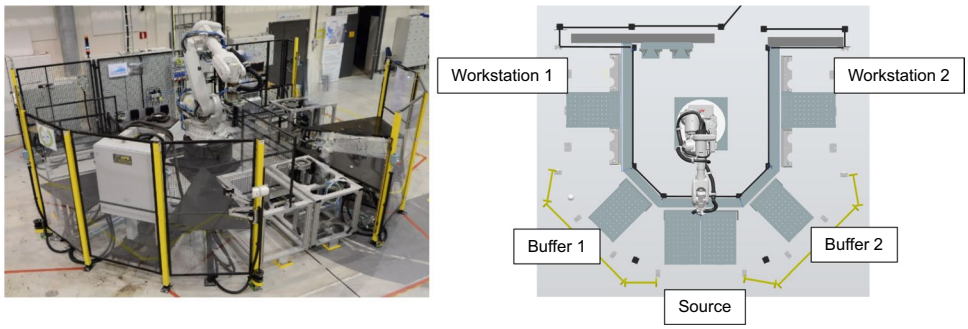

Fig. 3 The Plug & Produce cell used as a base for the case study (a) and the locations in the cell of the case study (b)

inspection respectively and both have the requirement that the part must be attached to their location before execution. The *robot* is modelled as resource *robot_transport* with specialized transport interface that has a high-risk skill and effects "human" type skills. The *worker*s in the cell are modelled as resources owning skill *inspection* that is of type "human". This specifies that the risk of robot transport only affect humans in the operational space of the skill. Also, the workers own skills to transfer the part between the assembly and inspection locations. In addition, *carriers* resources are available with skills to transport the part safely but at slower pace between the buffers to the workstations.

At runtime the part agent, selects the lowest-cost assembly skill among available *workstation_assembly*. Because assembly requires the part to be attached, the planner first negotiates a transport plan. The onNegotiation function contributes to identify the unsafe edges including skill *robot_transport* and returns the cost of the safe ones. When *robot_transport* is executed, onOverlap books the worker's interface before the move to prevent the hazardous spatial overlap. Inspection also needs the part attached, so recursive planning assigns *worker_transport*, after which the part is inspected and leaves the cell.

For each of the two system configurations, the planner constructs a map, represented as planning graphs in Fig. 4, to solve the planning problems associated with each requirement. For the *assembly* skill requirement.

- the initial state: $s_i = s_0$,
- in configuration A, the proposition stands: $s_0 = (Sr = part \wedge WS_{assembly1} = \phi)$,
- and in configuration B the proposition stands: $s_0 = (Sr = part \wedge (WS_{assembly1} = \phi \vee WS_{assembly2} = \phi))$.

Here.

- Sr denote the attachment status of the source location interface attachment,
- $WS_{assembly1}, WS_{assembly2}$ represent the attachment status of the workstation assembly interfaces,
- ϕ indicates free interface.

The goal states are defined as.

- Configuration A: $s_g = s_{12} = (WS_{assembly1} = part)$,
- Configuration B: $S_g = \begin{bmatrix} s_{12} = (WS_{assembly1} = part), \\ s_{22} = (WS_{assembly2} = part) \end{bmatrix}$.

For *inspection* skill requirement, the initial states are.

- In configuration A: $s_i = s_{12}$,
- And in configuration B $s_i = s_{12} \vee s_{22}$.

The corresponding goal states are $s_g = s_{13}, s_{23}$ where the propositions are.

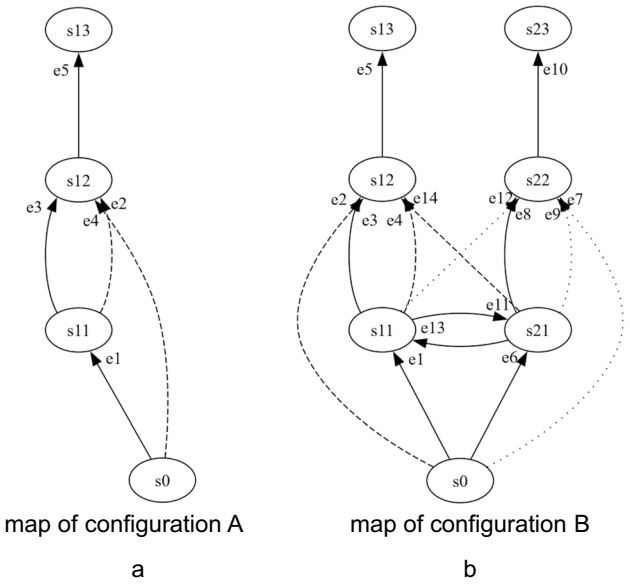

Fig. 4 The maps generated by the planner for each configuration, a: map of configuration A and b: map of configuration B. The dashed edges are excluded in case robot_transport to $\mathrm{WS}_{\mathrm{assembly1}}$ is unsafe, and the dotted edges are filtered out in case robot_transport to $\mathrm{WS}_{\mathrm{assembly2}}$ is unsafe

- in s_{13}: $WS_{inspection1} = part$,
- in s_{23} is $WS_{inspection2} = part$.

In the scenario, the cost of an edge considers the time required to complete the edge. Edges that include *robot_transport* and end at a workstation location are affected by the SSM policies, whereas the other edges have static transport time. Transport times are computed using the measured distances and speed constraints. Table 2 summarises the calculated transport times under both normal speed (500 mm/s) and reduced speed (150 mm/s) zones:

Complexity discussion

The map is constructed of edges. Let R be the number of transport resources, P the number of transported agents, and n_{rp} the number of locations compatible with resource r for transported agent p. The number of candidate edges before safety pruning is showed in (6)

$$E_{cand} = \sum_{r \in R} \sum_{p \in P} n_{rp}(n_{rp} - 1) \tag{6}$$

Consider x is the average of n_{rp} over all (r, p), then $E_{\text{cand}} \approx R\,P\,x(x-1) = O(RPx^2)$, so candidates grow linearly with resources and part types, and quadratically with average compatibility. Suppose a constant-time safety and cost negotiations (onNegotiation) for each skill of each edge, hence negotiation overhead scales linearly with the candidates: $O(E_{\text{cand}}) = O(RPx^2)$, with three checks per candidate.

Complexity of solving the subsequent planning problem depends on the solver and the generated problem model. The same map if applied to different solver classes will generate different complexity. Typically for the SAT solver class it is NP-complete, and the complexity can be expressed in worst case $O(2^b)$ and the b if the number of Boolean variables (Impagliazzo et al., 2001). In practice, our cell-level scope keeps b modest, which keeps total planning time small.

In an expansion trial with 24 location specialisation interfaces and one fully compatible transport resource, the candidate edges set is $24 \times 23 = 552$ edges, implying $\approx 3 \times 552 = 1{,}656$ negotiations. Map construction accounted for about 10 per cent of total planning time, and SAT planning remained at or below ~300 ms, indicating responsiveness at realistic cell scale.

Table 2 transport time (cost) for edges in the map

Edge	Transport time at 500 mm/s (s)	Transport time at 150 mm/s (s)	Robot independent transport time (s)
e1	5	–	
e2	8	27	
e3	–	–	10
e4	3	10	
e5	–	–	1
e6	5	–	
e7	8	27	
e8	–	–	10
e9	3	10	
e10	–	–	1
e11	10	–	
e12	13	44	
e13	10	–	
e14	13	44	

Simulation results and discussion

To evaluate the efficiency gains of the proposed planner, we conducted repeated simulations incorporating agent configurations, and associated edge costs. Performance of the proposed planning approach was compared against conditions where safety relied solely on SSM collaborative method. The evaluation considered three human-presence patterns; low, medium, and high. These patterns represented different operator behaviours, ranging from operators standing back to frequently entering the work zone. This design tests whether the proposed planner remains effective under changing human proximity patterns.

The cumulative cost and throughput were recorded for both configurations A and B under three human-presence patterns with 30 independent replications. Every replication executed the plans required for 200 consecutive parts. At each planning step, the human proximity state was sampled according to pattern-specific probabilities. Operator–robot proximity was discretised into three states consistent with SSM: safe distance ($d \geq 2\,\text{m}$), reduced speed ($1.2 \leq d < 2\,\text{m}$), and unsafe proximity ($d < 1.2\,\text{m}$). We defined the three representative human-presence patterns as probability distributions over these states: Low presence (0.85, 0.10, 0.05), Medium presence (0.60, 0.30, 0.10), and High presence (0.40, 0.40, 0.20). These distributions were chosen to capture a range of operator behaviours in compact, high-utilisation cells and to stress-test planner robustness, while allowing operators to move freely rather than follow fixed schedules.

The baseline corresponds to an SSM-only system without the proposed planner applied. The proposed method uses the same action set and optimal solver but adds safety negotiation. The choice of solver (e.g. Z3, MathSAT) is not critical, as all return the same optimal plan.

Throughput results are summarised in Table 3. The proposed planner consistently outperformed the baseline across both configurations. In Configuration A, throughput increased from 3.38 to 5.21 parts per 60 cost units, while

in Configuration B it rose from 3.29 to 5.89. The larger improvement in Configuration B demonstrates that the benefits of the planner scale with opportunities for parallel execution, which reactive safety alone cannot exploit.

Cumulative costs are presented in Tables 4 and 5 for Configurations A and B, respectively. In Configuration A, the proposed planner reduced costs by approximately 600 units under low human presence, 1,400 units under medium, and more than 2,500 units under high. Configuration B showed even greater reductions, ranging from about 870 units at low presence to nearly 2,900 units at high. These results indicate that the efficiency gains of proactive safety-aware planning increase as human interaction becomes more frequent.

Variability across replications was also markedly reduced. Under reactive safety, cost outcomes exhibited large standard deviations e.g.,±242 in Configuration A, high presence, while with the proposed planner, deviations narrowed to below±55. This confirms that the method not only improves mean performance but also enhances predictability. This is a critical feature for industrial deployment, where stable and reliable throughput is important.

Overall, the results demonstrate that the proposed planner provides significant improvements over SSM-only safety. It reduces execution costs, increases throughput, and delivers more consistent performance across scenarios. Importantly, the planner enables parallelism to become beneficial, whereas under reactive safety it offers no advantage except under low human-presence conditions. These findings confirm that considering safety constraints during plan generation is essential for achieving both safe and efficient operation in human-integrated smart manufacturing systems.

Building on these observations, we conducted a statistical analysis to verify that the observed gains are statistically significant. Table 6 reports the results of t-test comparisons for cumulative cost reductions between the proposed system and the SSM baseline. The mean difference Δ_{cost} (cost units) was defined as in (7).

$$\Delta_{\text{cost}} = mean(C_{SSM}) - mean(C_{proposed}) \tag{7}$$

where $mean(C)$ denotes the mean final cumulative cost over replications for producing 200 parts for each human pattern. Thus, $\Delta > 0$ indicates that the proposed system achieves a lower mean cost than SSM alone. In every human-presence pattern and in both configurations, the reductions are statistically significant.

Table 7 summarises the statistical analysis for throughput, averaged across all human-presence patterns. The mean difference Δ_{thr} (parts/60) was defined as in (8).

$$\Delta_{\text{thr}} = mean(T_{proposed}) - mean(T_{SSM}) \tag{8}$$

Table 3 Throughput results. Unit: parts per 60 cost units

Configuration	Proposed approach	SSM-only baseline
Configuration A	5.21±0.61	3.38±0.97
Configuration B	5.89±0.57	3.29±0.92

Table 4 cumulative cost—configuration A. Unit: cost units

Configuration A		
Human pattern	Planned/ SSM-only baseline	Mean cost±s.d
Low	Planned	2002.3±31.5
Low	SSM-only baseline	2603.2±147.6
Medium	Planned	2357.9±56.4
Medium	SSM-only baseline	3770.1±169.9
High	Planned	2635.4±52.8
High	SSM-only baseline	5171.4±242.2

Table 5 cumulative cost—configuration B. Unit: cost units

Configuration B		
Human pattern	Planned/SSM-only baseline	Mean cost±s.d
Low	Planned	1833.6±10.6
Low	SSM-only baseline	2703.7±183.3
Medium	Planned	2020.8±31.7
Medium	SSM-only baseline	3881.2±173.7
High	Planned	2318.3±43.1
High	SSM-only baseline	5209.3±249.0

Table 6 Statistical analysis of cumulative cost reductions

Configuration	Human-presence pattern	Δ_{cost} (cost units)	95% CI	t	p
A	Low	600.9	534.8–667.0	17.81	$<10^{-12}$
A	Medium	1412.2	1333.7–1490.7	35.28	$<10^{-12}$
A	High	2536.0	2427.4–2644.6	45.75	$<10^{-12}$
B	Low	870.1	789.6–950.6	21.19	$<10^{-12}$
B	Medium	1860.4	1783.0–1937.8	47.12	$<10^{-12}$
B	High	2891.0	2780.2–3001.8	51.16	$<10^{-12}$

where $mean(T)$ denotes the mean throughput (parts per 60 cost units) averaged across Low, Medium, and High patterns. A positive Δ indicates higher throughput with the proposed system.

From the statistical analysis tables, the 95% CI does not include 0 and p value is less than 0.001 significant level. Thus, the improvement is statistically significant and indicates strong evidence of a real gain. These results prove that the gains achieved by the proposed planner rather than from the experimental settings.

Analysis of variability significance confirmed that the proposed system produced significantly lower variability. Table 8 shows the analysis results of variability significance. Standard-deviation ratios (baseline/proposed) ranged from about 3× to 17× across scenarios, with all confidence intervals above 1 and all p values $\ll 0.001$. This indicates that the proposed approach yields not only lower mean costs but also more stable outcomes.

Table 7 Statistical analysis of throughput improvements

Configuration	Δ_{thr}(parts/60 cost unit)	95% CI	t	p
A	1.83	1.54–2.12	12.37	$<10^{-12}$
B	2.60	2.33–2.87	18.61	$<10^{-12}$

Table 8 Analysis of variability significance

Scenario	SD ratio (SD baseline/SD proposed)	95% CI	*p* value
A–low	4.69	2.99–7.35	$<10^{-12}$
A–medium	3.01	1.92–4.72	$<10^{-12}$
A–high	4.59	2.93–7.19	$<10^{-12}$
B–low	17.29	11.03–27.11	$<10^{-12}$
B–medium	5.48	3.50–8.59	$<10^{-12}$
B–high	5.78	3.69–9.06	$<10^{-12}$

Further, we discuss how results change if the proportion between skill cost is different. This is to generalise the results to other case e.g., stricter safety polices such as larger safety zones or lower robot speeds which increases the cost, faster robot speeds or looser zones reduces the cost, or different costs for non-effected skill cost by the safety polices.

Let an edge e include a robot transport and the following definitions.

- c_e: expected total cost of edge e.
- $c_{norm}, c_{red}, c_{stop}$: total cost of edge e conditional on the SSM state being safe distance i.e. normal speed, reduced separation i.e. reduced speed, or unsafe proximity i.e. protective stop, respectively. Each may include fixed detach and attach skill costs.
- $p_{\text{SD}}, p_{\text{RS}}, p_{\text{UP}}$: probabilities of those SSM states (Safe Distance, Reduced Speed, and Unsafe Proximity) at execution time, with $p_{\text{SD}} + p_{\text{RS}} + p_{\text{UP}} = 1$.

The probability-weighted edge cost under baseline SSM is given by (9)

$$c_e = p_{\text{SD}}\, c_{\text{norm}} + p_{\text{RS}}\, c_{\text{red}} + p_{\text{UP}}\, c_{\text{stop}} \tag{9}$$

The proposed planner reduces the costly states by $\Delta p_{\text{RS}}, \Delta p_{\text{UP}} \geq 0$ through safety-aware planning. Let α scales costs, the improvement over a reactive SSM baseline is given by (10)

$$\Delta C \;\approx\; \alpha([\Delta p_{\text{RS}}(c_{red} - c_{norm}) + \Delta p_{\text{UP}}(c_{stop} - c_{norm})]) \tag{10}$$

This shows that when α increases in cases of larger zones and lower allowable speeds, the magnitude of improvement scales up linearly with α. Also, the gains remain even with reduced α because $\Delta p_{\text{RS}}, \Delta p_{\text{UP}} \geq 0$ and $c_{\text{red}} > c_{\text{norm}}$, $c_{\text{stop}} > c_{\text{norm}}$. Also, in case human behaviour changes reducing $p_{\text{RS}}, p_{\text{UP}}$ the improvements are reduced but remain until the case where no human presence exists and $p_{\text{RS}} = p_{\text{UP}} = 0$ as expected.

Conclusion

This research was motivated by an emerging challenge in smart manufacturing; automated planners typically leave safety to be handled after plans are generated. Such reactive handling limits throughput, triggers avoidable stops in shared workspaces with humans, and undermines the flexibility that modern industry demands. We investigated whether runtime-evaluated safety policies could be negotiated by agents and embedded into MAS planning to improve safety and efficiency of transport and positioning plans. Specifically, we examined the integration of agent negotiation with a safety-aware ontology to proactively exclude unsafe skill sequences and dynamically assess execution costs that include safety measures. The hypothesis is that incorporating such proactive reasoning into the planning process will increase the generated plans' efficiency in human-integrated manufacturing environments. The contributions are threefold. First, the safety-aware C-MAS ontology to provide planning semantics for safe transport and positioning planning within smart manufacturing. Second, a planning approach was introduced that constructs enhanced edges, each representing a compact sequence of positioning and transportation skills with their aggregated execution cost. These edges are generated through runtime negotiation to ensure safety and feasibility under runtime conditions. Third, the planner was integrated with an execution method that maintains safety during concurrent skill execution through skills overlap detection and agents' interfaces booking. Evaluation in a Plug & Produce case study with two configurations and three human presence patterns demonstrated clear and consistent advantages over a reactive baseline. The approach consistently improved throughput, reduced execution costs, and delivered more stable performance under stochastic human presence. The implications extend beyond the case study and address broader needs in manufacturing. The approach shows particular promise for low-volume, high-mix production, where frequent reconfigurations and human interaction make reactive safety especially limiting. Future work will focus on enhancing the runtime replanning capabilities to handle cases where executing a previously safe edge or skill becomes unsafe. This includes revising the ontology to include monitoring and renegotiation strategies to adjust unsafe segments of the plan without discarding it fully which enhances planning outcome and safety under uncertainty. The study demonstrates that human-centric manufacturing benefits when

safety is treated as a planning factor rather than only as a runtime constraint. Instead of enclosing workers within fenced areas, it safeguards them by intelligently reasoning about their presence.

Author contributions The study conception and design were performed by Bassam Massouh and Fredrik Danielsson. Material preparation and data collection were performed by Bassam Massouh. Analysis was performed by Bassam Massouh. The original draft of the manuscript was written by Bassam Massouh. Fredrik Danielsson and Sudha Ramasamy reviewed and edited the original draft. All authors read and approved the final manuscript.

Funding Open access funding provided by University West. This work was supported by KK-foundation Dnr. 20230032.

Data availability No Data associated in the manuscript.

Declarations

Competing Interests The authors declare that there are no competing financial or non-financial interests or personal relationships that are directly or indirectly related to this work.

References

Alkazzi, J.-M., & Okumura, K. (2024). A comprehensive review on leveraging machine learning for multi-agent path finding. *IEEE Access, 12*, 57390–57409. https://doi.org/10.1109/ACCESS.2024.3392305

Arai, T., Izawa, H., Maeda, Y., Kikuchi, H., Ogawa, H., & Sugi, M. (2003). Real-time task decomposition and allocation for a multi-agent robotic assembly cell. *Proceedings of the IEEE International Symposium on Assembly and Task Planning, 2003*, 42–47. https://doi.org/10.1109/ISATP.2003.1217185

Bi, M., Kovalenko, I., Tilbury, D. M., & Barton, K. (2024). Dynamic distributed decision-making for resilient resource reallocation in disrupted manufacturing systems. *International Journal of Production Research, 62*(5), 1737–1757. https://doi.org/10.1080/00207543.2023.2200567

Blum, A. L., & Furst, M. L. (1997). Fast planning through planning graph analysis. *Artificial Intelligence, 90*(1–2), 281–300. https://doi.org/10.1016/S0004-3702(96)00047-1

Botea, A., Enzenberger, M., Mueller, M., & Schaeffer, J. (2005). Macro-FF: Improving AI planning with automatically learned macro-operators. *Journal of Artificial Intelligence Research, 24*, 581–621. https://doi.org/10.1613/jair.1696

Byner, C., Matthias, B., & Ding, H. (2019). Dynamic speed and separation monitoring for collaborative robot applications—Concepts and performance. *Robotics and Computer-Integrated Manufacturing, 58*, 239–252. https://doi.org/10.1016/j.rcim.2018.11.002

Choudhury, S., Gupta, J. K., Kochenderfer, M. J., Sadigh, D., & Bohg, J. (2022). Dynamic multi-robot task allocation under uncertainty and temporal constraints. *Autonomous Robots, 46*(1), 231–247. https://doi.org/10.1007/s10514-021-10022-9

Coles, A. I., & Smith, A. J. (2007). Marvin: A heuristic search planner with online macro-action learning. *Journal of Artificial Intelligence Research, 28*, 119–156. https://doi.org/10.1613/jair.2077

Deng, Y., Chow, A. H. F., Yan, Y., Su, Z., Zhou, Z., & Kuo, Y.-H. (2025). Hierarchical production control and distribution planning under retail uncertainty with reinforcement learning. *International Journal of Production Research, 63*(12), 4504–4522. https://doi.org/10.1080/00207543.2025.2452386

Farid, A. M., & Ribeiro, L. (2015). An axiomatic design of a multiagent reconfigurable mechatronic system architecture. *IEEE Transactions on Industrial Informatics, 11*(5), 1142–1155. https://doi.org/10.1109/TII.2015.2470528

Gao, C., He, X., Dong, H., Liu, H., & Lyu, G. (2022). A survey on fault-tolerant consensus control of multi-agent systems: Trends, methodologies and prospects. *International Journal of Systems Science, 53*(13), 2800–2813. https://doi.org/10.1080/00207721.2022.2056772

Ghallab, M., Nau, D., & Traverso, P. (2016). *Automated planning and acting*. Cambridge University Press.

Gottardi, A., Terreran, M., Frommel, C., Schoenheits, M., Castaman, N., Ghidoni, S., & Menegatti, E. (2023). Dynamic human-aware task planner for human-robot collaboration in industrial scenario. *European Conference on Mobile Robots (ECMR), 2023*, 1–8. https://doi.org/10.1109/ECMR59166.2023.10256268

Gunes, V., Peter, S., Givargis, T., & Vahid, F. (2014). A survey on concepts, applications, and challenges in cyber-physical systems. *KSII Transactions on Internet and Information Systems, 8*(12), 4242–4268. https://doi.org/10.3837/tiis.2014.12.001

Hanna, A., Larsson, S., Götvall, P.-L., & Bengtsson, K. (2022). Deliberative safety for industrial intelligent human–robot collaboration: Regulatory challenges and solutions for taking the next step towards industry 4.0. *Robotics and Computer-Integrated Manufacturing, 78*, Article 102386. https://doi.org/10.1016/j.rcim.2022.102386

Heuss, L., Gebauer, D., & Reinhart, G. (2024). Concept for the automated adaption of abstract planning domains for specific application cases in skills-based industrial robotics. *Journal of Intelligent Manufacturing, 35*(8), 4233–4258. https://doi.org/10.1007/s10845-023-02211-3

Hu, Y., Wang, Y., Hu, K., & Li, W. (2023). Adaptive obstacle avoidance in path planning of collaborative robots for dynamic manufacturing. *Journal of Intelligent Manufacturing, 34*(2), 789–807. https://doi.org/10.1007/s10845-021-01825-9

Impagliazzo, R., Paturi, R., & Zane, F. (2001). Which problems have strongly exponential complexity? *Journal of Computer and System Sciences, 63*(4), 512–530. https://doi.org/10.1006/jcss.2001.1774

Ju, Y., Ding, D., He, X., Han, Q.-L., & Wei, G. (2022). Consensus control of multi-agent systems using fault-estimation-in-the-loop: Dynamic event-triggered case. *IEEE/CAA Journal of Automatica Sinica, 9*(8), 1440–1451. https://doi.org/10.1109/JAS.2021.1004386

Kagermann, H., Helbig, J., Hellinger, A., & Wahlster, W. (2013). Recommendations for implementing the strategic initiative INDUSTRIE 4.0: Securing the future of German manufacturing industry; final report of the Industrie 4.0 Working Group. Forschungsunion.

Kaiser, J., McFarlane, D., Hawkridge, G., André, P., & Leitão, P. (2023). A review of reference architectures for digital manufacturing:

Classification, applicability and open issues. *Computers in Industry, 149*, Article 103923. https://doi.org/10.1016/j.compind.2023.103923

Kavvathas, K., Kampourakis, E., Andronas, D., Fousekis, N., & Makris, S. (2024). A service-oriented orchestration and planning tool for plug and produce manufacturing: A deformable object handling supervision paradigm. *International Journal of Computer Integrated Manufacturing, 37*(12), 1626–1649. https://doi.org/10.1080/0951192X.2024.2331533

Kheirabadi, M., Keivanpour, S., Chinniah, Y. A., & Frayret, J.-M. (2023). Human-robot collaboration in assembly line balancing problems: Review and research gaps. *Computers & Industrial Engineering, 186*, Article 109737. https://doi.org/10.1016/j.cie.2023.109737

Kootbally, Z., Schlenoff, C., Antonishek, B., Proctor, F., Kramer, T., Harrison, W., Downs, A., & Gupta, S. (2018). Enabling robot agility in manufacturing kitting applications. *Integrated Computer-Aided Engineering, 25*(2), 193–212. https://doi.org/10.3233/ICA-180566

Kovalenko, I., Balta, E. C., Tilbury, D. M., & Barton, K. (2023). Cooperative product agents to improve manufacturing system flexibility: A model-based decision framework. *IEEE Transactions on Automation Science and Engineering, 20*(1), 440–457. https://doi.org/10.1109/TASE.2022.3156384

Lee, E. A. (2008). Cyber Physical systems: Design challenges. In 2008 11th IEEE international symposium on object and component-oriented real-time distributed computing (ISORC), 363–369. https://doi.org/10.1109/ISORC.2008.25

Leitão, P. (2009). Agent-based distributed manufacturing control: A state-of-the-art survey. *Engineering Applications of Artificial Intelligence, 22*(7), 979–991. https://doi.org/10.1016/j.engappai.2008.09.005

Lyu, G., & Brennan, R. W. (2024). Evaluating a self-manageable architecture for industrial automation systems. *Robotics and Computer-Integrated Manufacturing, 85*, Article 102627. https://doi.org/10.1016/j.rcim.2023.102627

Ma, W., Duan, A., Lee, H.-Y., Zheng, P., & Navarro-Alarcon, D. (2025). Human-aware reactive task planning of sequential robotic manipulation tasks. *IEEE Transactions on Industrial Informatics, 21*(4), 2898–2907. https://doi.org/10.1109/TII.2024.3514130

Marzia, S., Vital-Soto, A., & Azab, A. (2023). Automated process planning and dynamic scheduling for smart manufacturing: A systematic literature review. *Manufacturing Letters, 35*, 861–872. https://doi.org/10.1016/j.mfglet.2023.07.013

Massouh, B., Danielsson, F., Lennartson, B., Ramasamy, S., & Khabbazi, M. (2024). Safe and reconfigurable manufacturing: Safety aware multi-agent control for plug & produce system. *The International Journal of Advanced Manufacturing Technology, 134*(1–2), 529–544. https://doi.org/10.1007/s00170-024-14112-7

Merckaert, K., Convens, B., Nicotra, M. M., & Vanderborght, B. (2024). Real-time constraint-based planning and control of robotic manipulators for safe human–robot collaboration. *Robotics and Computer-Integrated Manufacturing, 87*, Article 102711. https://doi.org/10.1016/j.rcim.2023.102711

Nilsson, A., Danielsson, F., & Svensson, B. (2023). From CAD to plug & produce: A generic structure for the integration of standard industrial robots into agents. *International Journal of Advanced Manufacturing Technology, 128*(11–12), 5249–5260. https://doi.org/10.1007/s00170-023-12280-6

Onori, M., Lohse, N., Barata, J., & Hanisch, C. (2012). The IDEAS project: Plug & produce at shop-floor level. *Assembly Automation, 32*(2), 124–134. https://doi.org/10.1108/01445151211212280

Papavasileiou, A., Aivaliotis, S., Glykos, C., Koukas, S., & Makris, S. (2024). An AI-based decision-making framework with task planning and dynamic reconfiguration capabilities. In European Robotics Forum 2024. ERF 2024, 313–318. https://doi.org/10.1007/978-3-031-76428-8_58

Pupa, A., & Secchi, C. (2021). A safety-aware architecture for task scheduling and execution for human-robot collaboration. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021*, 1895–1902. https://doi.org/10.1109/IROS51168.2021.9636855

Qian, J., Zhang, Z., Shi, L., & Song, D. (2023). An assembly timing planning method based on knowledge and mixed integer linear programming. *Journal of Intelligent Manufacturing, 34*(2), 429–453. https://doi.org/10.1007/s10845-021-01819-7

Qu, W., Li, J., Zhang, R., Liu, S., & Bao, J. (2024). Adaptive planning of human–robot collaborative disassembly for end-of-life lithium-ion batteries based on digital twin. *Journal of Intelligent Manufacturing, 35*(5), 2021–2043. https://doi.org/10.1007/s10845-023-02081-9

Radakovič, M., Obitko, M., & Mařík, V. (2012). Dynamic explicitly specified behaviors in distributed agent-based industrial solutions. *Journal of Intelligent Manufacturing, 23*(6), 2601–2621. https://doi.org/10.1007/s10845-011-0593-6

Ribeiro, L., & Hochwallner, M. (2018). On the design complexity of cyberphysical production systems. *Complexity*. https://doi.org/10.1155/2018/4632195

Rizk, Y., Awad, M., & Tunstel, E. W. (2018). Decision making in multiagent systems: A survey. *IEEE Transactions on Cognitive and Developmental Systems, 10*(3), 514–529. https://doi.org/10.1109/TCDS.2018.2840971

Rogalla, A., Fay, A., & Niggemann, O. (2018). Improved domain modeling for realistic automated planning and scheduling in discrete manufacturing. In 2018 IEEE 23rd international conference on emerging technologies and factory automation (ETFA), 464–471. https://doi.org/10.1109/ETFA.2018.8502631

Scholz, C., Cao, H.-L., Imrith, E., Roshandel, N., Firouzipouyaei, H., Burkiewicz, A., Amighi, M., Menet, S., Sisavath, D. W., Paolillo, A., Rottenberg, X., Gerets, P., Cheyns, D., Dahlem, M., Ocket, I., Genoe, J., Philips, K., Stoffelen, B., Van den Bosch, J., & Vanderborght, B. (2025). Sensor-enabled safety systems for human-robot collaboration: A review. *IEEE Sensors Journal, 25*(1), 65–88. https://doi.org/10.1109/JSEN.2024.3496905

Shen, W., Wang, L., & Hao, Qi. (2006). Agent-based distributed manufacturing process planning and scheduling: A state-of-the-art survey. *IEEE Transactions on Systems, Man, and Cybernetics. Part C, Applications and Reviews, 36*(4), 563–577. https://doi.org/10.1109/TSMCC.2006.874022

Shibuya, T., Endo, T., & Matsuno, F. (2023). Experimental investigation of distributed navigation and collision avoidance for a robotic swarm. *Artificial Life and Robotics, 28*(1), 50–61. https://doi.org/10.1007/s10015-022-00843-x

SIS (Swedish Standards Institute). (2016). Robots and robotic devices—Collaborative robots, (ISO/TS 15066:2016, IDT). www.sis.se

SIS (Swedish Standards Institute). (2025). Robots and robotic devices—Safety requirements for industrial robots—Part 1: Robots (ISO 10218-1:2025). www.sis.se

Tsutsumi, D., Bergmann, J., Dobrovoczki, P., Hayashi, N., Kovács, A., Szádoczki, Z., Szaller, Á., & Umeda, S. (2023). Dynamic production line re-balancing by alternative plans for compensating equipment failures. *Procedia CIRP, 120*, 810–815. https://doi.org/10.1016/j.procir.2023.09.080

Vogel-Heuser, B., Lee, J., & Leitão, P. (2015). Agents enabling cyber-physical production systems. *At - Automatisierungstechnik, 63*(10), 777–789. https://doi.org/10.1515/auto-2014-1153

Wang, D., Zhao, J., Han, M., & Li, L. (2025). Deep reinforcement learning-based energy-aware disassembly planning for end-of-life products with stimuli-activated self-disassembly. *Journal of*

Intelligent Manufacturing, 36(8), 5475–5494. https://doi.org/10.1007/s10845-024-02527-8

Zhang, D., Feng, G., Shi, Y., & Srinivasan, D. (2021). Physical safety and cyber security analysis of multi-agent systems: A survey of recent advances. *IEEE/CAA Journal of Automatica Sinica, 8*(2), 319–333. https://doi.org/10.1109/JAS.2021.1003820

Zhao, R., Tao, S., & Li, P. (2025). Safety-efficiency integrated assembly: The next-stage adaptive task allocation and planning framework for human–robot collaboration. *Robotics and Computer-Integrated Manufacturing, 94*, 102942. https://doi.org/10.1016/j.rcim.2024.102942

Zhou, W., Chen, D., Yan, J., Li, Z., Yin, H., & Ge, W. (2022). Multi-agent reinforcement learning for cooperative lane changing of connected and autonomous vehicles in mixed traffic. *Autonomous Intelligent Systems, 2*, 5. https://doi.org/10.1007/s43684-022-00023-5